D007

Runtime Framework Architecture

D007 defines the complete runtime execution environment of Echo Engine. It establishes the layer responsible for creating, managing, executing, scheduling, timing, and shutting down runtime operations. The document is architecture only and becomes the canonical reference for all future runtime implementation.

Field	Summary
Objective	Define the runtime backbone that coordinates execution and lifecycle control for the engine.
Scope	Runtime lifecycle, runtime host, kernel, clock, scheduler, context, configuration, events, session management, statistics, and state management.
Non-scope	Assets, ECS, rendering, physics, and gameplay behavior are consumers or adjacent systems, not runtime policy owners.
Outcome	A dependency-injected runtime framework with explicit boot, execution, and shutdown behavior.

The runtime layer is the engine execution backbone. If the framework is the structural core, the runtime is the operational core. It must be deterministic, observable, and safe to compose into different host scenarios without changing the meaning of its contracts.

1. Runtime Mandate

The runtime is the execution environment that turns the framework into an active engine.

The runtime exists to own execution. It starts after framework composition, manages active runtime services, and shuts down in a controlled reverse order. It does not replace the framework. It consumes the framework and adds time, scheduling, sessions, and frame-oriented control.

D007 is intentionally strict about responsibility. If runtime begins to own asset policy, rendering policy, or gameplay policy, it will stop being a clean execution layer and become a hidden application layer. The runtime must therefore remain focused on coordination, not domain behavior.

Its job is to answer a narrow set of questions at all times: is the runtime ready, what is it currently doing, what is the current frame or session state, and how should dependent systems be advanced or stopped.

Runtime rules:

• own execution, timing, lifecycle, scheduling, and runtime state

• remain independent of asset ownership and feature-domain policy

• expose state and statistics in a structured, inspectable form

• keep all dependencies injected through composition rather than hidden globally

• treat shutdown as a first-class phase, not an afterthought

The runtime is therefore the engine?s active operational contract. Later milestones may extend it, but they must not redefine its ownership boundaries.

2. Scope and Directory Boundaries

The runtime directory defines the canonical runtime architecture.

The runtime layer lives under src/framework/runtime/ and is intentionally organized by responsibility. Core runtime classes sit at the root. Errors, events, pipeline, session, state, and statistics live in dedicated folders so that the execution model remains readable and modular.

This directory structure is a contract. If a later milestone needs runtime behavior, it should attach to the existing runtime seams rather than creating parallel runtime systems elsewhere in the codebase.

File	Role
RuntimeClock.js	Owns runtime timing source, frame time model, and tick timing semantics.
RuntimeConfiguration.js	Owns runtime configuration inputs, precedence, and resolved settings.
RuntimeContext.js	Owns the runtime execution context and injected runtime-scoped services.
RuntimeHost.js	Owns the host-facing runtime wrapper and lifecycle integration boundary.
RuntimeKernel.js	Owns the runtime orchestration, initialization, update, and shutdown flow.
RuntimeScheduler.js	Owns scheduling policy and frame advancement coordination.

Subdirectories separate responsibility families: errors/, events/, pipeline/, session/, state/, and statistics/. That separation keeps the runtime composable and makes it clear which parts are policy, which are data, and which are execution mechanics.

3. Dependency Model

Runtime dependencies flow inward through composition and outward through controlled contracts.

The dependency chain begins with the composition root. The runtime builder assembles a runtime host. The host owns a kernel. The kernel owns clock, scheduler, context, and runtime service coordination. Support subsystems such as events, sessions, state, and statistics are then injected into the kernel through explicit contracts.

This direction exists because runtime policy must be centralized. If lower-level components can create their own clocks or schedulers, the engine will lose determinism. If sessions or events bypass the kernel, the runtime stops being an orchestrated environment and becomes a collection of unrelated helpers.

Layer	Owns	Reason
CompositionRoot	Builds runtime object graph.	Keeps wiring in one place.
RuntimeBuilder	Assembles runtime host and dependencies.	Owns runtime construction policy.
RuntimeHost	Host-facing runtime boundary.	Owns runtime lifecycle entry and exit.
RuntimeKernel	Orchestrates runtime execution.	Owns runtime state and phase control.
RuntimeClock	Time source and tick model.	Defines runtime timing semantics.
RuntimeScheduler	Frame advancement policy.	Controls ordered execution.
RuntimeContext	Injected runtime-scoped services and data.	Makes the runtime environment explicit.
RuntimeStatistics	Runtime health and performance observations.	Reports without owning policy.

The direction exists to preserve a single source of truth for runtime behavior. The kernel is the only place where runtime-wide orchestration is allowed to converge.

4. Boot Process

Application startup flows from framework composition into runtime activation.

The boot path must be deterministic and easy to inspect. The application starts in the outer host, enters Bootstrap, receives the framework, passes through CompositionRoot, and then uses RuntimeBuilder to construct RuntimeHost. RuntimeHost then activates RuntimeKernel, which starts runtime services and transitions to the ready state.

Each step has one purpose. The boot sequence should be understandable without reading implementation code because it is a contract for ownership, not a procedural trick.

Step	Meaning
Application	Creates the host process or editor shell and decides to start the engine.
Bootstrap	Hands startup control to the framework layer in a controlled form.
Framework	Provides composition, service wiring, and engine-level control.
CompositionRoot	Binds concrete runtime dependencies and selected policies.
RuntimeBuilder	Constructs runtime host and runtime context objects.
RuntimeHost	Owns the runtime surface exposed to the framework.
RuntimeKernel	Initializes runtime services and begins frame control.
Runtime services	Enter active runtime participation through the kernel.
Runtime ready	Runtime state is prepared for execution and frame updates.

The boot sequence must remain one-directional. Anything that tries to re-enter composition after runtime activation risks recursive startup, duplicated services, or ambiguous ownership.

5. Shutdown Process

Runtime shutdown must be reverse-ordered, explicit, and fully observable.

Shutdown is a lifecycle phase, not an emergency escape hatch. The runtime should stop sessions, drain pipeline work, halt the scheduler, freeze updates, close event dispatch, release runtime services, and finally clear clock and context-owned state in a controlled order.

Reverse dependency destruction is important because later layers depend on earlier infrastructure. If the runtime clears a clock before stopping the scheduler, or clears sessions before draining pending events, it risks leaks, half-finished work, or incorrect final statistics.

Phase	Shutdown rule
Session cleanup	Stop active sessions and transition them to a terminal state.
Pipeline shutdown	Drain or cancel pending frame-stage execution.
Clock shutdown	Stop timing sources and freeze frame progression.
Scheduler shutdown	Disable frame advancement and ordered dispatch.
Runtime service shutdown	Stop runtime-scoped services in reverse-safe order.
Kernel cleanup	Clear runtime-owned references and emit final status.

The shutdown path should produce a final diagnostic summary so that later milestone implementations can understand where the runtime stopped and what resources were still in flight.

RuntimeKernel

RuntimeKernel is the central runtime orchestrator.

Aspect	Architecture guidance
Purpose	Own runtime execution control, initialization, update flow, and shutdown coordination.
Responsibilities	Advance states, coordinate clock and scheduler, manage runtime services, and enforce lifecycle rules.
Ownership	Owns runtime control flow and active runtime state, not feature behavior.
Dependencies	Depends on RuntimeHost, RuntimeClock, RuntimeScheduler, RuntimeContext, events, state, and statistics.
Lifetime	Created by the host and lives for the duration of runtime execution.
Public API	Initialize, start, update, pause, resume, shutdown, and runtime status methods.
What it owns	Lifecycle state, active orchestration, and runtime-phase decisions.
What it never owns	Asset loading, ECS policy, rendering policy, or gameplay semantics.

RuntimeHost

RuntimeHost is the host-facing wrapper around the runtime kernel.

Aspect	Architecture guidance
Purpose	Expose the runtime to the framework and outer application while preserving separation.
Responsibilities	Build kernel dependencies, present host-level lifecycle entry points, and coordinate host shutdown.
Ownership	Owns host/runtime boundary behavior, not runtime policy itself.
Dependencies	Depends on RuntimeConfiguration, RuntimeContext, kernel, and diagnostics.
Lifetime	Exists from runtime construction through final host shutdown.
Public API	Create, attach, initialize, execute, stop, and dispose semantics at a high level.
What it owns	The runtime host boundary and host-visible lifecycle state.
What it never owns	Feature services, asset databases, or feature-specific scheduling.

RuntimeClock

RuntimeClock owns the timing model used by the runtime.

Aspect	Architecture guidance
Purpose	Provide a stable source of time, delta, frame count, and tick progression.
Responsibilities	Measure runtime time, expose timing snapshots, and support deterministic tick semantics.
Ownership	Owns the timing source abstraction and time state.
Dependencies	Depends on runtime state and statistics only through explicit contracts.
Lifetime	Lives for the duration of runtime execution and freezes on shutdown.
Public API	Start, stop, tick, elapsed, delta, and frame time summary methods.
What it owns	Frame time, accumulated time, and timing policy state.
What it never owns	Scheduling order, session behavior, or feature timing policy.

RuntimeScheduler

RuntimeScheduler coordinates ordered frame advancement.

Aspect	Architecture guidance
Purpose	Schedule runtime execution phases and frame progression.

Aspect	Architecture guidance
Responsibilities	Choose when and in what order frame work is executed, while respecting the clock and state.
Ownership	Owns scheduling order and frame-step policy.
Dependencies	Depends on RuntimeClock, RuntimeState, RuntimePipeline, and events.
Lifetime	Operates during runtime execution and stops before final cleanup.
Public API	Schedule, advance, pause, resume, and order-control methods.
What it owns	Frame ordering policy and scheduler state.
What it never owns	Feature-specific work or time source implementation.

RuntimeContext

RuntimeContext carries runtime-scoped services and state into execution.

Aspect	Architecture guidance
Purpose	Provide an explicit runtime execution envelope for injected runtime services.
Responsibilities	Hold runtime-scoped references, host-provided services, and selection results.
Ownership	Owns runtime-scoped context data, not global engine state.
Dependencies	Depends on configuration, session selection, runtime host, and framework services.
Lifetime	Created during boot and disposed during shutdown.
Public API	Read-only context accessors and runtime-scoped service queries.
What it owns	Injected runtime context data and runtime-visible service handles.
What it never owns	Feature state, asset memory, or domain logic.

RuntimeConfiguration

RuntimeConfiguration resolves runtime inputs into a stable policy snapshot.

Aspect	Architecture guidance
Purpose	Represent the final runtime configuration after precedence and validation.
Responsibilities	Merge configuration sources, validate values, and expose immutable runtime settings.
Ownership	Owns runtime policy values only.
Dependencies	Depends on framework configuration, host inputs, session policy, and diagnostics.
Lifetime	Resolved at startup and read throughout runtime execution.
Public API	Resolve, inspect, validate, and query policy views.
What it owns	Resolved configuration values and validation state.
What it never owns	Live mutable settings bags or feature-specific policy objects.

RuntimeStatistics

RuntimeStatistics collects coarse-grained runtime observability data.

Aspect	Architecture guidance
Purpose	Provide runtime-level health and performance information.
Responsibilities	Track frame timing, session counts, pipeline health, and lifecycle milestones.
Ownership	Owns observed metrics and snapshots only.
Dependencies	Depends on clock, scheduler, state, sessions, events, and diagnostics.
Lifetime	Sampling starts with runtime execution and ends with shutdown.
Public API	Snapshot, summary, counters, and diagnostic-ready metric access.

Aspect	Architecture guidance
What it owns	Runtime observations, counts, and summarized metrics.
What it never owns	Policy decisions or feature telemetry pipelines.

Runtime statistics should answer whether the runtime is healthy, how it is behaving per frame, and where bottlenecks might exist. They should not become a replacement for a profiler.

6. Error Subsystem

Runtime errors are structured, hierarchical, and diagnostic-first.

Runtime errors are not throwaway messages. They are part of the architecture contract. Each runtime error type exists to preserve ownership, identify the layer that failed, and guide recovery or shutdown behavior. The hierarchy should support precise classification without encouraging ad hoc branching.

Error propagation must preserve context. A host error may wrap a kernel error. A kernel error may wrap a configuration or session error. The runtime should surface the most relevant failure while retaining enough detail for diagnostics and tooling.

Error type	Role
RuntimeError	Base runtime failure contract for all runtime-specific errors.
RuntimeHostError	Represents host-layer runtime failures and composition problems.
RuntimeKernelError	Represents runtime orchestration and execution failures.
RuntimeInitializationError	Represents boot and initialization failures.
RuntimeConfigurationError	Represents invalid or inconsistent runtime configuration.
RuntimeSessionError	Represents session lifecycle and selection failures.
RuntimeStateError	Represents invalid state transitions or lifecycle violations.
MissingDependencyError	Represents unmet runtime dependency requirements.

Recovery philosophy: detect early, classify accurately, stop cleanly when a dependency or state requirement is broken, and preserve enough context for the next implementation layer to fix the problem without guesswork.

7. Event Subsystem

Runtime events define the controlled signal path for runtime coordination.

The runtime event system is the runtime-side communication layer. It carries runtime lifecycle changes, frame state, session updates, scheduler notifications, and diagnostic signals. It should remain immutable once published and should always be created through explicit dependency injection paths.

The event subsystem exists to decouple producers and consumers without hiding ownership. Immediate dispatch is for low-latency state changes. Deferred dispatch is for frame-safe or ordered processing. Queueing must remain visible and bounded by runtime policy.

Class	Role
RuntimeEvent	Immutable runtime event envelope with type, source, payload, and timing context.
RuntimeEvents	Event topic vocabulary used by the runtime architecture.
RuntimeEventBus	Subscription and routing infrastructure for runtime events.
RuntimeEventDispatcher	Delivery coordinator for immediate and deferred event handling.

Event ownership stays with the runtime system that creates the event. Event lifetime ends when delivery and any scheduled retention policy are complete. The bus and dispatcher must not mutate event meaning.

Dependency injection is mandatory here because event routing is a runtime policy surface. A global event singleton would destroy the separation D007 is meant to preserve.

8. Pipeline Subsystem

The frame pipeline defines ordered runtime stages and their isolation rules.

The frame pipeline turns runtime execution into a sequence of well-defined stages. Each stage has a narrow responsibility, and the pipeline enforces ordering so that state transitions, event dispatch, session work, and statistics collection happen in a predictable sequence.

Stage isolation matters because runtime subsystems should not be able to skip ahead or reach backward into a stage that is not yet complete. That keeps update order deterministic and makes failure handling far easier to reason about.

Class	Role
FramePipeline	Owns the ordered execution graph for runtime frame processing.
FrameStage	Represents a single isolated phase in the pipeline.
PipelineResult	Represents success, failure, or deferred completion for a stage or frame.

Pipeline execution should always respect the same stage ordering unless a future milestone introduces a formally approved stage extension. Failure in one stage should short-circuit subsequent dependent stages rather than letting the frame continue in an inconsistent state.

9. Session Subsystem

Runtime sessions provide the unit of active runtime participation.

A runtime session is the structured unit of activity inside the runtime framework. It lets the engine identify a selection, activation, and destruction path for the current runtime mode. Sessions make it possible for later milestone implementations such as editor runtime, dedicated server, or simulation runtime to share the same runtime backbone while selecting different active behavior.

The registry owns the available session definitions. The session manager owns selection and activation. The factory creates sessions from descriptors. The lifecycle model keeps the session state explicit so that runtime can switch or tear down cleanly.

Class	Role
RuntimeSession	An active runtime participation unit with explicit lifecycle state.
RuntimeSessionFactory	Creates runtime sessions from descriptors and policy inputs.
RuntimeSessionRegistry	Catalog of available session descriptors and capabilities.
RuntimeSessionManager	Selection, activation, transition, and destruction coordinator.
RuntimeSessionLifecycle	Lifecycle vocabulary for session creation, activation, and teardown.
SessionDescriptor	Metadata describing a session?s purpose, requirements, and compatibility.
SessionSelectionPolicy	Rules for choosing which session should activate in a given runtime mode.

Selection policy should be deterministic. If two sessions are eligible, the policy must define how the runtime decides. If no session is eligible, the runtime must produce a structured failure rather than guessing.

10. State Subsystem

Runtime state makes lifecycle transitions explicit and reviewable.

Runtime state is the runtime?s contract with itself. It describes whether the runtime is constructed, initializing, ready, executing, paused, shutting down, or completed. The state machine is the only approved path for moving between these states.

Validation must be strict. Illegal transitions should fail fast and should include enough context for diagnostics to explain the violation. State recovery is possible only when the runtime can safely move to a valid terminal or quiescent state.

Class	Role
RuntimeState	Immutable or controlled snapshot of the runtime?s current lifecycle state.
RuntimeStateMachine	Owns transition policy and validation rules.
RuntimeTransition	Represents a named movement between two runtime states.

State ownership stays with the kernel and state machine. Other subsystems may observe state, but they should not mutate it directly. That rule prevents accidental lifecycle drift and keeps failures local.

11. Runtime Statistics

Statistics observe runtime health without becoming policy.

Runtime statistics collect frame metrics, memory-adjacent snapshots, timing summaries, and lifecycle counters. They support diagnostics, profiling handoff, and operational visibility, but they do not decide behavior. The runtime must be able to expose its health without tangling measurement with control logic.

Collection philosophy: measure coarse-grained outcomes that help identify runtime regressions and system stress, not every micro-event. The runtime framework is the right place to observe frame timing, session churn, pipeline failures, and shutdown integrity.

Topic	Rule
Collection philosophy	Collect coarse, stable, lifecycle-relevant metrics.
Performance metrics	Track initialization cost, frame cadence, and pipeline pressure.
Frame metrics	Record frame time, delta summaries, and stage completion status.
Memory metrics	Expose high-level memory pressure or ownership observations.
Runtime diagnostics	Emit structured runtime health snapshots to diagnostics systems.

Statistics should be read-only from the outside. If a later milestone needs richer telemetry, it should extend the runtime diagnostics contract without reassigning ownership of the runtime statistics layer.

12. Complete Runtime Lifecycle

Construction, dependency injection, initialization, execution, update, shutdown, and cleanup are the full runtime lifecycle.

The runtime lifecycle begins when the host constructs the runtime graph and continues through dependency injection, initialization, execution, frame updates, and shutdown. Cleanup finalizes the contract and clears runtime-owned references so the next host cycle can begin cleanly.

The lifecycle pattern is important because runtime is long-lived compared to many other engine services. It must stay stable under repeated frame advancement while still allowing a clean, deterministic exit.

Phase	Meaning
Construction	Runtime objects are assembled but not yet active.
Dependency injection	Required services and policy objects are bound explicitly.
Initialization	Clock, scheduler, context, sessions, and state are prepared.
Execution	Runtime services become active and respond to scheduling.
Frame update	The pipeline and scheduler advance one frame at a time.
Shutdown	Sessions, pipeline, scheduler, clock, and services stop in reverse order.
Cleanup	Runtime-owned references are cleared and final diagnostics are emitted.

A runtime lifecycle that skips explicit shutdown is incomplete. D007 treats cleanup as part of correct operation, not as optional polishing.

13. Integration with Engine Subsystems

The runtime coordinates subsystems without taking ownership of their domain logic.

The runtime interacts with framework services, assets, animation, rendering, world and scene structures, ECS, physics, gameplay, networking, editor workflows, save systems, streaming, and diagnostics. Its role is to schedule, activate, and shutdown participation, not to define the rules of those systems.

The architecture should remain symmetrical: subsystems may observe runtime state, subscribe to runtime events, and register through runtime sessions or services, but they should not replace runtime policy with their own hidden timing model.

Subsystem	Interaction rule
Framework	Owns composition; runtime consumes the composed framework services.
Assets	Runtime requests loaded assets and handles lifecycle visibility.
Animation	Runtime advances animation services through scheduler and pipeline.
Rendering	Runtime triggers render-facing update opportunities but does not own rendering policy.
World	Runtime hosts world progression through frame stages or sessions.
Scene	Runtime can activate scene-oriented sessions and lifecycle hooks.
ECS	Runtime may schedule ECS updates without owning ECS behavior.
Physics	Runtime advances physics participation in ordered frame stages.
Gameplay	Runtime supplies frame cadence and state, not gameplay rules.
Networking	Runtime may coordinate deterministic updates and network ticks.
Editor	Runtime supports editor runtime and simulation modes through session policy.
Save System	Runtime provides stable lifecycle and state checkpoints.
Streaming	Runtime coordinates streaming-ready frame timing and event order.
Diagnostics	Runtime exposes state, statistics, and failure detail to diagnostics.

The runtime should be the common timing and lifecycle contract that these systems share. It should never become the place where they are redefined.

14. Architectural Decisions and Tradeoffs

Runtime design choices are deliberate responses to common engine mistakes.

D007 chooses an explicit runtime kernel, host boundary, clock, scheduler, context, and state machine because those separations make execution behavior easier to reason about. A common mistake in engine design is to bury runtime behavior in a generic update loop without ownership boundaries; another is to let every subsystem manage its own timing. D007 avoids both.

The tradeoff is a slightly larger number of runtime classes. That is acceptable because the classes are intentionally narrow and because runtime clarity is more valuable than compact but ambiguous code.

Principle	Why it exists
Dependency Injection	All runtime services and timing policy objects must be supplied explicitly.
Composition Root	The runtime object graph is built in one deliberate place.
SOLID	Each runtime class owns one slice of runtime responsibility.
Single Responsibility	Clock clocks, scheduler schedules, kernel orchestrates, host hosts.
Loose Coupling	Subsystems interact through runtime contracts and events, not hidden state.
High Cohesion	Related runtime behavior stays in runtime folders and classes.
One Class Per File	Each runtime concept remains reviewable and freezeable on its own.
Private Fields	Runtime implementation state should stay internal by default.
ES Modules	Keeps dependency direction visible and import order explicit.
No Global Mutable State	Eliminates hidden coupling and unstable runtime behavior.
No Circular Imports	Maintains a clean module graph and deterministic composition.
Registry Pattern	Used for sessions and event topics where discovery matters.
Factory Pattern	Used where session or runtime object creation is policy-driven.
Lifecycle Pattern	Used to formalize runtime boot, execution, and shutdown phases.

The most important separation is between scheduling and state. A scheduler decides when work runs. A state machine decides whether it may run at all.

15. Extensibility and Future Milestone Integration

Future runtime features should extend the existing seams rather than creating parallel infrastructure.

D007 is designed to be extended by later milestones such as multithreading, a job system, a fiber scheduler, dedicated server mode, remote runtime, editor runtime, simulation runtime, and headless runtime. The extension rule is that new behavior should plug into the existing host, kernel, scheduler, session, event, and pipeline seams.

This keeps the runtime canonical. Future runtime variants may alter policy, but they should not change the meaning of the runtime contracts or break the established lifecycle.

Future area	How it plugs in	Constraint
Multithreading	Add concurrency through scheduled runtime services.	Kernel and scheduler remain the coordination point.
Job System	Attach background work to runtime stages.	Jobs must respect runtime state and timing rules.
Fiber Scheduler	Introduce cooperative execution policy.	Must fit inside runtime scheduling contracts.
Dedicated Server	Select server-oriented session and runtime policy.	No change to runtime ownership model.
Remote Runtime	Support remote execution or orchestration.	Context and host abstractions stay intact.
Editor Runtime	Use editor-specific session and runtime selection.	Same kernel, different selected policy.
Simulation Runtime	Expose simulation-focused frame semantics.	State machine and pipeline stay canonical.
Headless Runtime	Run runtime without presentation layer dependencies.	Runtime must remain platform-independent.

If a future runtime feature cannot be expressed as a session, scheduler, pipeline stage, event source, or state transition, it likely belongs in a later architecture layer rather than in D007.

16. Verification and Freeze Criteria

D007 is ready when the runtime can be explained completely through its contracts.

The runtime architecture should be frozen only when reviewers can trace the boot sequence, update sequence, and shutdown sequence without consulting implementation code. The document must describe what each class owns, what it never owns, and how it interacts with every other runtime subsystem.

Verification should focus on dependency direction, lifecycle clarity, error propagation, event immutability, and session/state consistency. The goal is to confirm that the runtime can serve as the canonical implementation target for future work.

Architecture verification checklist

• RuntimeKernel, RuntimeHost, RuntimeClock, RuntimeScheduler, RuntimeContext, RuntimeConfiguration, RuntimeStatistics each have one clear ownership boundary

• error hierarchy is specific enough to identify host, kernel, configuration, session, and state failures

• event ownership, dispatch, queueing, and immutability are described explicitly

• pipeline execution order and stage isolation are clear

• session creation, activation, registry ownership, and destruction are defined

• runtime state transitions are validated and recoverable where appropriate

• statistics remain observational and do not become policy

Dependency verification checklist

• composition root and runtime builder are the only approved construction path

• runtime modules do not create their own hidden clocks or schedulers

• events, sessions, state, and statistics remain runtime dependencies rather than external globals

• import graphs remain acyclic and readable

Integration verification checklist

• framework integration is visible and stable

• asset, animation, rendering, ECS, physics, gameplay, networking, editor, save, and streaming integrations are all expressed as runtime consumers

• shutdown order is reverse-safe and resource cleanup is explicit

Freeze checklist

• all runtime classes are accounted for and scoped

• lifecycle phases are complete and consistent

• timing, scheduling, and state ownership are unambiguous

• the runtime can be used as the canonical reference for implementation without reinterpretation

Review checklist

• reviewers can explain the boot path from application to ready runtime

• reviewers can explain why the kernel owns orchestration and the host owns the boundary

• reviewers can explain how future milestones extend runtime without replacing it

D007 is frozen when the runtime can be summarized as a deterministic, dependency-injected execution environment with explicit scheduling, timing, sessions, state, and diagnostics.

17. Canonical Summary

The runtime contract in one short statement for future implementers.

Echo Engine D007 defines the runtime framework: RuntimeHost exposes the runtime boundary, RuntimeKernel owns orchestration, RuntimeClock owns timing, RuntimeScheduler owns frame order, RuntimeContext and RuntimeConfiguration make runtime inputs explicit, RuntimeSessionManager and RuntimeStateMachine manage lifecycle and state, RuntimeEventBus and RuntimeEventDispatcher coordinate runtime signaling, RuntimeStatistics observe runtime health, and the pipeline organizes execution stages. The runtime owns execution, not domain behavior.

Future implementation AIs must treat this document as authoritative. If a later design conflicts with D007, the architecture specification wins unless it is formally revised.

Rule	Statement
Canonical rule	Prefer explicit runtime ownership over implicit update behavior.
Canonical ownership	Kernel orchestrates, host boundary exposes, and support systems stay specialized.
Canonical boundary	Runtime coordinates timing and lifecycle; feature systems consume it.
Canonical outcome	A canonical runtime backbone ready for implementation.

This closes the D007 architecture specification.